BoostMySkills — Database & Migration: Executive Summary

Commit `8e36e6d` **· 2026-07-22 · PostgreSQL 16.14 (UTC)**

The architecture, in one picture

BoostMySkills runs on **thirteen PostgreSQL tables**: **eleven** core application/learning/platform tables, **one** supporting account-lifecycle table (`account_deletion_requests`), and **one** operational bookkeeping table (`schema_migrations`). Identity, catalogue, enrolment, progress, assessment and certification are relational; the *shape of a course* lives in JSON.

Why the table count is small

We keep **relationships** relational (who is enrolled, who passed, who was certified) and put the **content tree** — sections, units, questions and answers — inside validated JSON on `credential_versions` . That avoids a sprawl of `sections` / `units` / `questions` / `assets` tables while keeping transactional integrity where it matters. Correctness (roles, statuses, one-attempt rule, single-draft/single-published rule, one-certificate-per-enrolment) is enforced by the database, not just the app.

Why content uses JSON

A course is a deeply nested, frequently-restructured tree, and each unit type (video, reading, PDF, MCQ) has a different shape. JSON absorbs new types and fields without a migration, while a strict contract validates every write and keeps **learner content and answer keys in two separate documents** — answers never leak.

Why credential revisions exist

Every credential has immutable **revisions**: at most one draft and one published at a time. Publishing never rewrites history, and learners stay bound to the revision they enrolled on — so updating a course cannot silently change what a current learner is studying or was certified against.

How users and history migrate

The live site is the **source**. Content moves via **OLX** (export → validate → import as draft → review → publish). **Users, enrolments, progress, grades and certificates need database/API exports in addition to OLX** — OLX alone does not contain them. Source user ids map to `external_ref` ; Clerk identities are created in the **target** environment (never copied from source/UAT); passwords follow an approved reset/invitation or hash-import path once feasibility is confirmed.

How OLX is used

OLX is the content bridge for both the one-time live migration and ongoing UAT→Production promotion. The importer already enforces archive safety (traversal, size bombs, symlinks, unsafe XML/HTML) and preserves source usage keys so historical progress can later attach to the right units.

Key migration risks

Password-hash compatibility with Clerk (unconfirmed); duplicate emails/usernames (reported, never merged); source usage keys that don't resolve to imported units (progress fallback to course-level); legacy certificate verification-code conflicts; unsupported XBlocks (must be reviewed, never silently dropped).

What is ready vs. what still needs decisions

- **Ready now:** the full schema and integrity rules; OLX import/export with safety hardening; backup and restore-verify tooling; a learner-import dry-run scaffold that refuses to fabricate results.

- **Still required:** live Open edX database/exports and version; the Clerk password strategy; the duplicate-user, verification-code, and unsupported-block policies; and a real UAT→Production promotion. **No migration has been executed, and none is claimed.**

Detail: `current-database-explained.md`, `current-database-uml.svg`, `live-openedx-to-current-platform.md`.